

Fraud Transaction Detection

Detecting Target Leakage, Comparing Seven Classifiers, and Cost-Sensitive Threshold Optimization

1. Introduction and Objective

This project builds and evaluates machine learning models to detect fraudulent transactions in a synthetic transaction dataset. Seven supervised classifiers, Logistic Regression, K-Nearest Neighbors, Decision Tree, Random Forest, Support Vector Machine, Naive Bayes, and AdaBoost, are trained and compared to predict a binary target, Fraud_Label (1 = fraud, 0 = legitimate).

What distinguishes this project from a standard classification exercise is a central discovery made during exploratory data analysis: two of the input features can, on their own, near-perfectly reconstruct the target label. Rather than ignore this or quietly work around it, the entire workflow is restructured around it. Every model in this report is trained and evaluated twice: once on the full feature set (referred to throughout as the naive pipeline), and once with the two leaking features removed (the leakage-aware pipeline). Comparing the two is the main analytical contribution of this work, since it separates genuine predictive skill from a shortcut baked into how the data was generated.

Beyond the leakage investigation, the project also covers class-imbalance handling, precision-recall analysis, a cost-sensitive threshold optimization tied to real transaction dollar amounts, and model explainability using SHAP.

2. Dataset Description

The dataset (fraud_transactions.csv) contains 50,000 transaction records across 21 columns: 20 input features plus the target column Fraud_Label. There are no missing values anywhere in the dataset.

- **Identifiers:** Transaction_ID, User_ID (dropped before modeling, no predictive meaning)
- **Numeric features:** Transaction_Amount, Account_Balance, Daily_Transaction_Count, Avg_Transaction_Amount_7d, Failed_Transaction_Count_7d, Card_Age, Transaction_Distance, Risk_Score
- **Categorical features:** Transaction_Type, Device_Type, Location, Merchant_Category, Card_Type, Authentication_Method (all low-cardinality, 3 to 5 unique values each)
- **Binary/flag features:** IP_Address_Flag, Previous_Fraudulent_Activity, Is_Weekend
- **Datetime:** Timestamp (a near-unique value per row, engineered rather than used directly, see Section 4)
- **Target:** Fraud_Label (1 = fraud, 0 = not fraud). Fraud rate: 32.13%, a moderate imbalance, not the severe 1-2% imbalance seen in many fraud datasets.

3. Exploratory Data Analysis

The target balance, and fraud rates broken down by Merchant_Category and Authentication_Method, are shown below. No single category shows a dramatic split, fraud is fairly evenly spread across merchant categories and authentication methods.



Figure 1. Target class balance (left), fraud by merchant category (middle), and fraud by authentication method (right).

Distributions of four representative numeric features, split by label, are shown next. `Transaction_Amount` and `Account_Balance` show little visible separation between fraud and non-fraud; `Risk_Score` and `Failed_Transaction_Count_7d`, by contrast, show a strikingly different distribution for the fraud class, a first hint of the leakage investigated in Section 5.

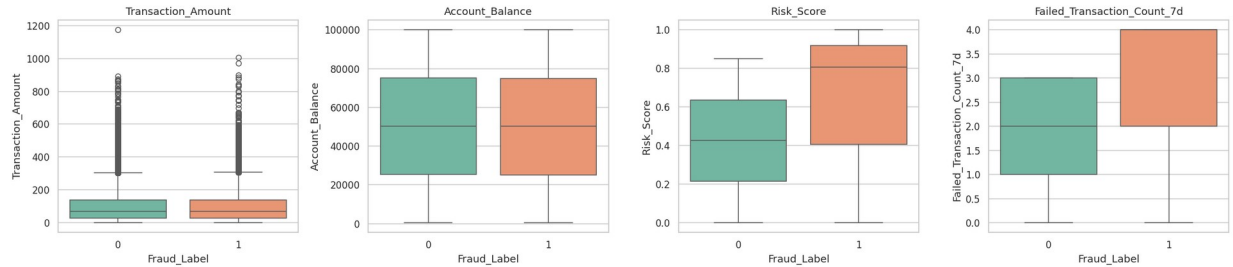


Figure 2. Boxplots of `Transaction_Amount`, `Account_Balance`, `Risk_Score`, and `Failed_Transaction_Count_7d`, split by `Fraud_Label`.

The correlation heatmap of all numeric features against each other and against the target confirms this. Every feature has a correlation with `Fraud_Label` below 0.01 in magnitude, except two: `Failed_Transaction_Count_7d` (0.51) and `Risk_Score` (0.39), both far larger than anything else in the dataset.

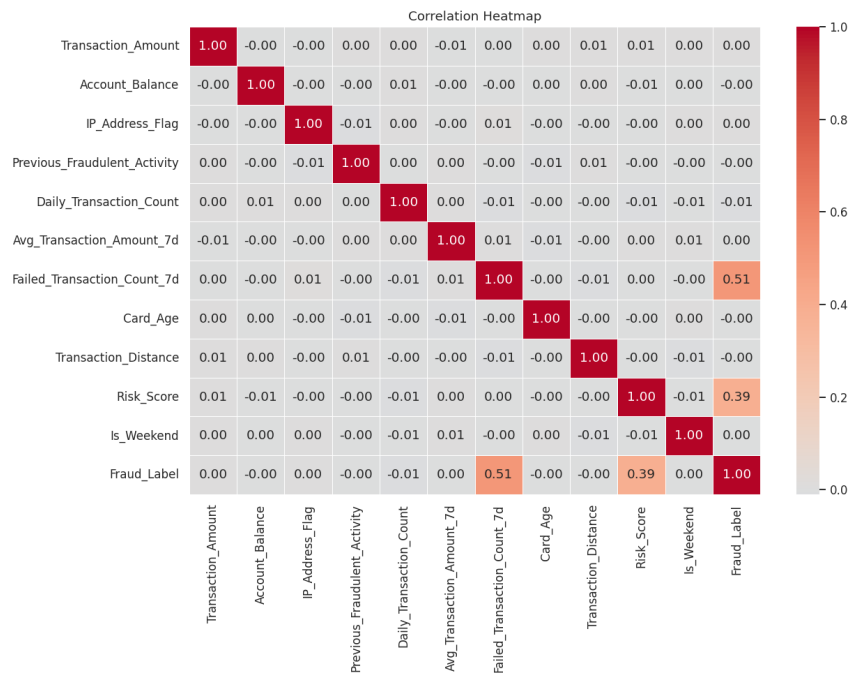


Figure 3. Correlation heatmap of numeric features and the target.

4. Leakage Investigation

A correlation of 0.39 to 0.51 is unusually strong for a single feature in a realistic fraud dataset, so this was investigated directly rather than accepted at face value. Grouping the fraud rate by `Failed_Transaction_Count_7d` reveals the following:

Failed_Transaction_Count_7d	Fraud Rate
0	15.4%
1	15.4%
2	14.5%
3	15.7%
4	100.0%

Every single record with Failed_Transaction_Count_7d equal to 4 is fraud, with no exceptions, out of 9,954 such records. A simple hand-written rule was then tested on the full dataset:

```
leak_rule = (Failed_Transaction_Count_7d == 4) OR (Risk_Score > 0.85)
```

This two-feature rule achieves 100.00% accuracy across all 50,000 rows. Among the records where Failed_Transaction_Count_7d is below 4, every fraud case has Risk_Score above 0.85 and every non-fraud case has Risk_Score at or below 0.85, a clean, non-overlapping split. The scatter plot below shows this separation directly: fraud (red) and non-fraud (blue) points do not overlap on either side of the Risk_Score = 0.85 line, except within the Failed_Transaction_Count_7d = 4 column, which is fraud in its entirety.

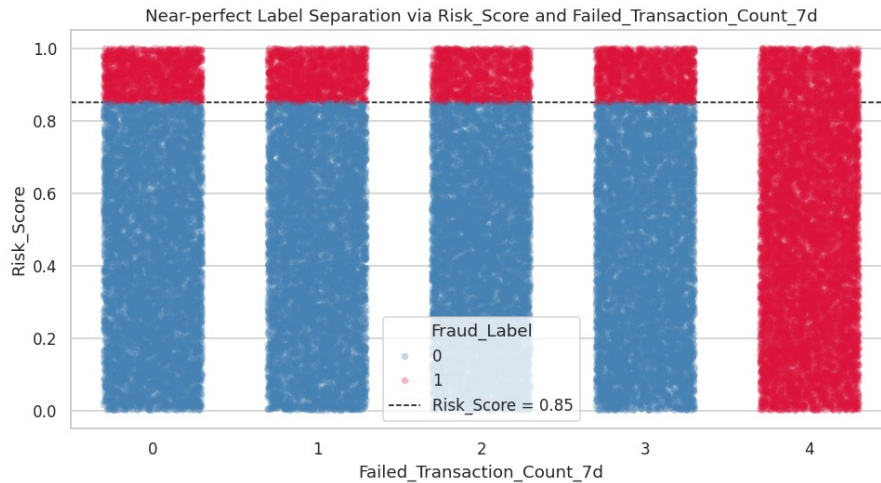


Figure 4. Risk_Score vs. Failed_Transaction_Count_7d, colored by Fraud_Label. The separation is essentially perfect.

This is target leakage: the two features were almost certainly used, directly or indirectly, to generate the label during the dataset's synthetic construction. Any model given access to these two features is not learning behavioral fraud patterns, it is rediscovering this rule. To make this explicit and quantifiable rather than a footnote, every model in this project is trained twice, on a naive feature set that includes both columns, and on a leakage-aware feature set with both columns removed.

5. Data Cleaning and Feature Engineering

- **Identifiers removed:** Transaction_ID and User_ID are dropped; they are unique per row and carry no generalizable signal.
- **Timestamp engineered, not encoded directly:** Timestamp is near-unique (47,724 distinct values across 50,000 rows in a comparable prior dataset), so label-encoding it directly would effectively assign each row an arbitrary large integer, noise rather than signal. It is instead parsed into Hour (0 to 23) and DayOfWeek (0 to 6), the components that could plausibly carry a real behavioral pattern.
- **Leak columns isolated:** Risk_Score and Failed_Transaction_Count_7d are kept in the naive feature set and explicitly removed to build the leakage-aware feature set, using the same underlying dataframe and row split for both, so the only difference between the two pipelines is the presence or absence of these two columns.

After cleaning, the naive feature set has 19 columns and the leakage-aware feature set has 17.

6. Train/Test Split and Preprocessing

The data is split 70/30 into training and test sets using a stratified split on Fraud_Label, so both sets preserve the 32.13% fraud rate. Critically, one shared split (by row index) is generated once and reused for both the naive and leakage-aware feature sets, so the only difference between the two pipelines' results is the feature set itself, not a different train/test partition.

Preprocessing is fit only on the training data and then applied to the test data, never fit on the full dataset, to avoid any information about the test set's category frequencies or numeric scale leaking into preprocessing:

- **Numeric features:** standardized with StandardScaler (mean 0, unit variance)

- **Categorical features:** one-hot encoded with `OneHotEncoder(handle_unknown='ignore', sparse_output=False)`, so a category that only appears in the test split is safely encoded as all-zero rather than crashing the pipeline, and the output is guaranteed to be a dense array (required downstream by SHAP).

After encoding, the naive pipeline expands to 38 columns and the leakage-aware pipeline to 36 columns (35,000 training rows and 15,000 test rows in both cases).

7. Modeling Framework

Seven classifiers are evaluated: Logistic Regression, K-Nearest Neighbors, Decision Tree, Random Forest, Support Vector Machine, Naive Bayes, and AdaBoost. Rather than duplicate per-model training code, a small reusable framework trains and scores every model in a single function, applied once to the naive feature set and once to the leakage-aware feature set.

- **Class imbalance handling:** `class_weight='balanced'` is applied to every model that supports it natively in scikit-learn: Logistic Regression, Decision Tree, Random Forest, and SVM. AdaBoost, Naive Bayes, and KNN have no such parameter.
- **Regularization:** Decision Tree is constrained to `max_depth=10`, `min_samples_leaf=20`; Random Forest to `max_depth=10`, `min_samples_leaf=10`. Left unconstrained, Random Forest was found to grow trees averaging depth ~ 38 on the leakage-aware features (verified directly), fitting noise rather than signal, and this also made exact SHAP explanation impractically slow.
- **Scoring for ranking metrics:** predicted probability is used where available (`predict_proba`); SVM uses its decision function directly rather than enabling the slower `probability=True` calibration.
- **Metrics computed per model:** accuracy, precision, recall, F1 (all for the fraud/positive class specifically, not a class-averaged blend), ROC-AUC, and average precision (PR-AUC). PR-AUC is reported throughout alongside ROC-AUC because it is the more honest metric when the positive class is a minority.

8. Pipeline A: Naive Results (All Features)

With both leaking features present, results are as follows:

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression	0.796	0.647	0.804	0.717	0.894	0.813
KNN	0.818	0.791	0.589	0.675	0.885	0.737
Decision Tree	1.000	1.000	1.000	1.000	1.000	1.000
Random Forest	1.000	1.000	1.000	1.000	1.000	1.000
SVM	0.975	0.933	0.994	0.963	0.999	0.998
Naive Bayes	0.886	0.840	0.795	0.817	0.944	0.897
AdaBoost	1.000	1.000	1.000	1.000	1.000	1.000

Three of seven models (Decision Tree, Random Forest, AdaBoost) achieve a literal, perfect 1.000 across every metric. This is not a sign of exceptional modeling, it is these tree-based models trivially rediscovering the two-feature rule from Section 4 in one or two splits. Confusion matrices, ROC curves, and precision-recall curves for all seven models are shown below.

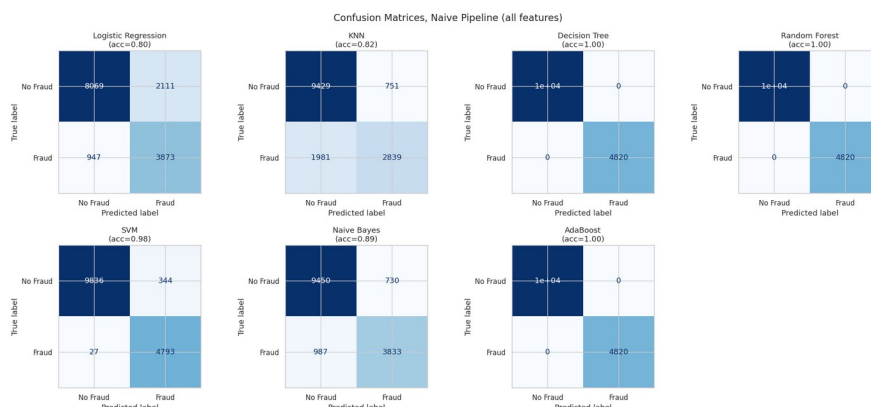


Figure 5. Confusion matrices, naive pipeline, all seven models.

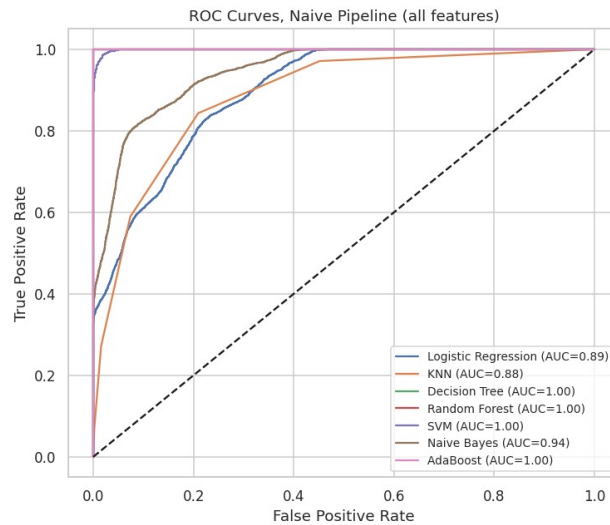


Figure 6. ROC curves, naive pipeline.

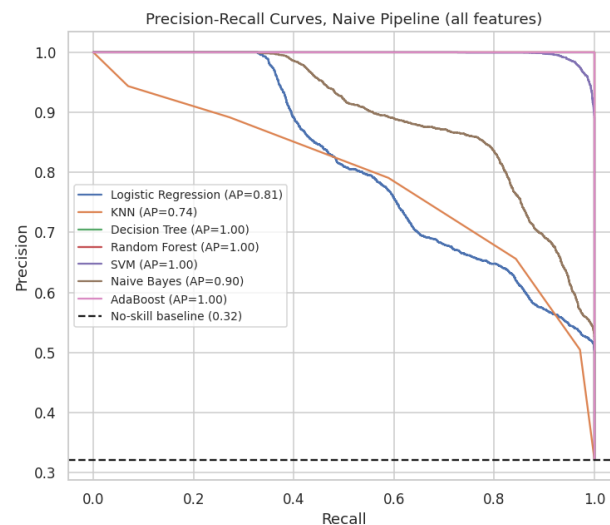


Figure 7. Precision-recall curves, naive pipeline.

9. Pipeline B: Leakage-Aware Results

With Risk_Score and Failed_Transaction_Count_7d removed, and every other aspect of the pipeline (split, scaling, encoding, class weighting) held identical, results collapse across the board:

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression	0.505	0.321	0.485	0.386	0.496	0.320
KNN	0.612	0.327	0.197	0.246	0.506	0.325
Decision Tree	0.477	0.317	0.545	0.401	0.497	0.321
Random Forest	0.572	0.318	0.288	0.302	0.497	0.320
SVM	0.509	0.317	0.460	0.376	0.493	0.317
Naive Bayes	0.679	0.000	0.000	0.000	0.494	0.320
AdaBoost	0.679	0.000	0.000	0.000	0.500	0.321

Every model's ROC-AUC falls between 0.493 and 0.506, statistically indistinguishable from 0.50, pure chance. Every model's PR-AUC (0.317 to 0.325) sits at essentially the no-skill baseline of 0.3213 (the dataset's own fraud rate). Naive Bayes and AdaBoost collapse entirely to predicting the majority class for every single test record, hence their 0.000

precision, recall, and F1. This is the central empirical finding of this project: once the two leak-derived features are removed, this synthetic dataset contains no learnable fraud signal at all, in any of the seven algorithms tested.

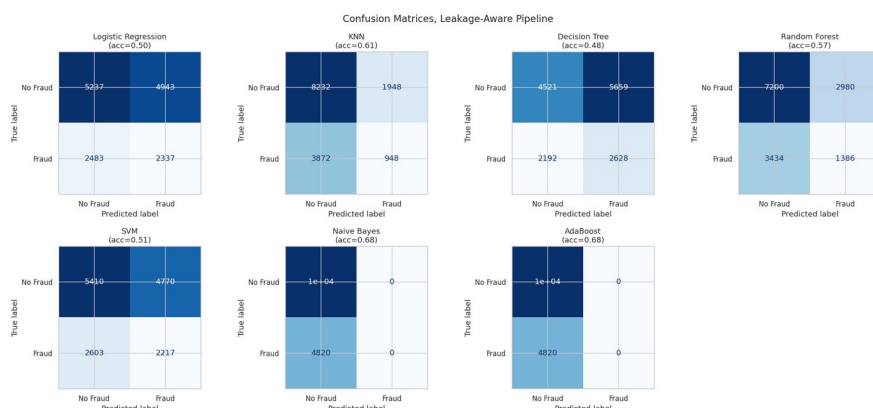


Figure 8. Confusion matrices, leakage-aware pipeline. Note how differently these are shaped compared to Figure 5.

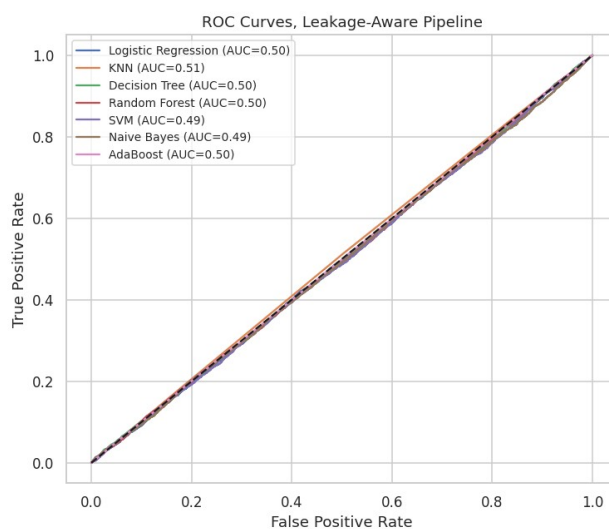


Figure 9. ROC curves, leakage-aware pipeline: every curve sits on the diagonal chance line.

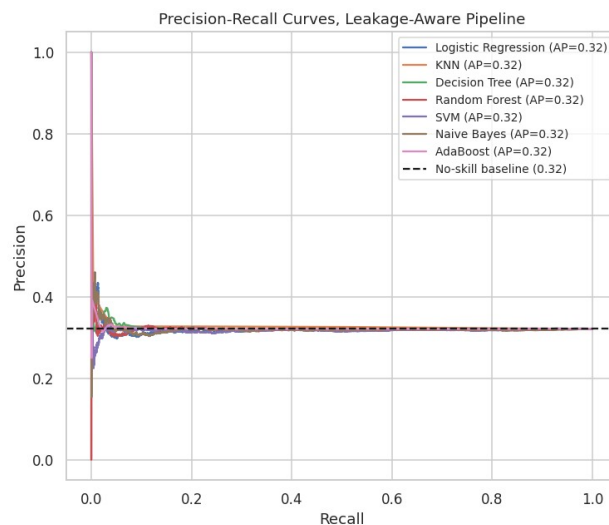


Figure 10. Precision-recall curves, leakage-aware pipeline: every curve sits at the no-skill baseline.

10. Naive vs. Leakage-Aware Comparison

The chart below is the single most important comparison in this project: it places the naive (leaky) and leakage-aware ROC-AUC and PR-AUC side by side for all seven models, making explicit how much of each model's apparent skill was the leak, versus real signal.

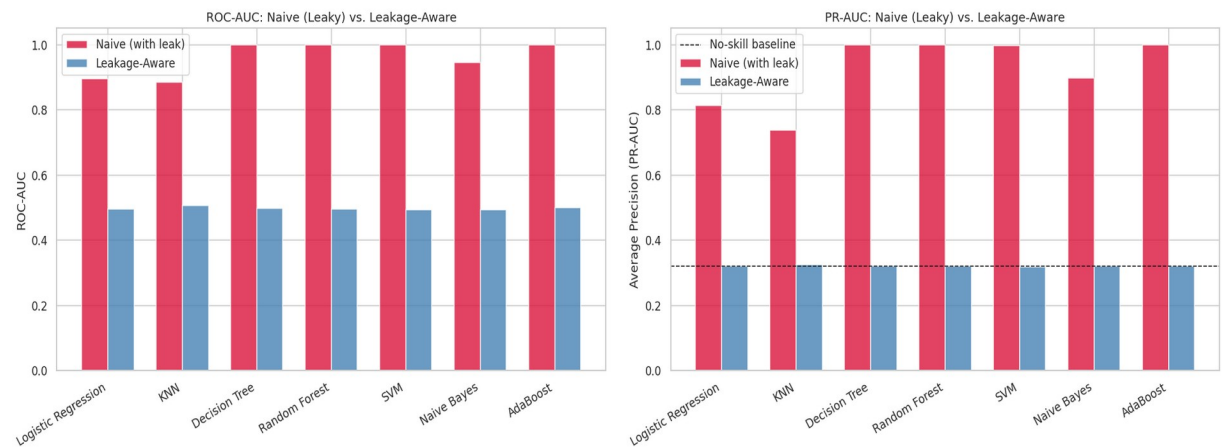


Figure 11. ROC-AUC (left) and PR-AUC (right): naive vs. leakage-aware, all seven models.

The same contrast, viewed through accuracy specifically (one uniquely-colored bar per model, value labeled), and through every metric at once as a single heatmap, tells the identical story:

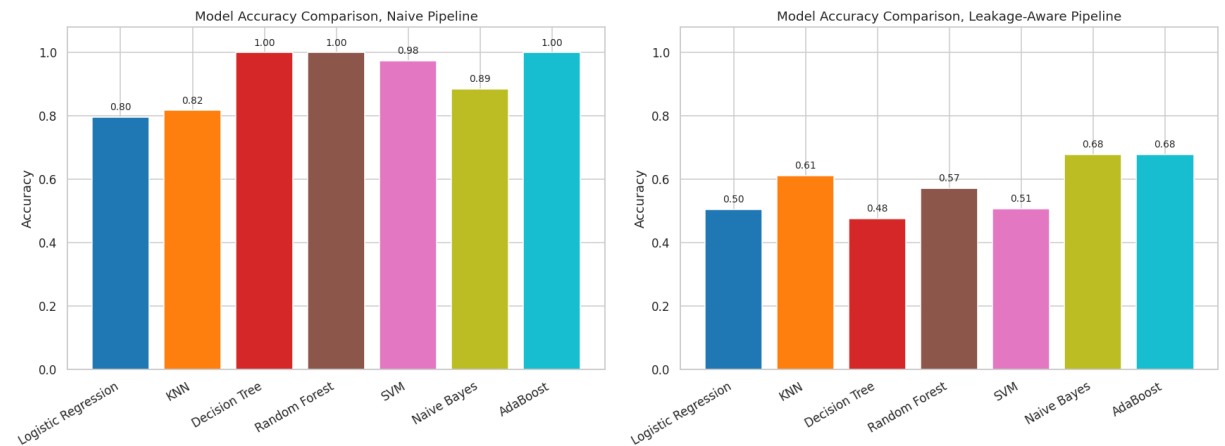


Figure 12. Accuracy by model, naive pipeline (left) vs. leakage-aware pipeline (right).

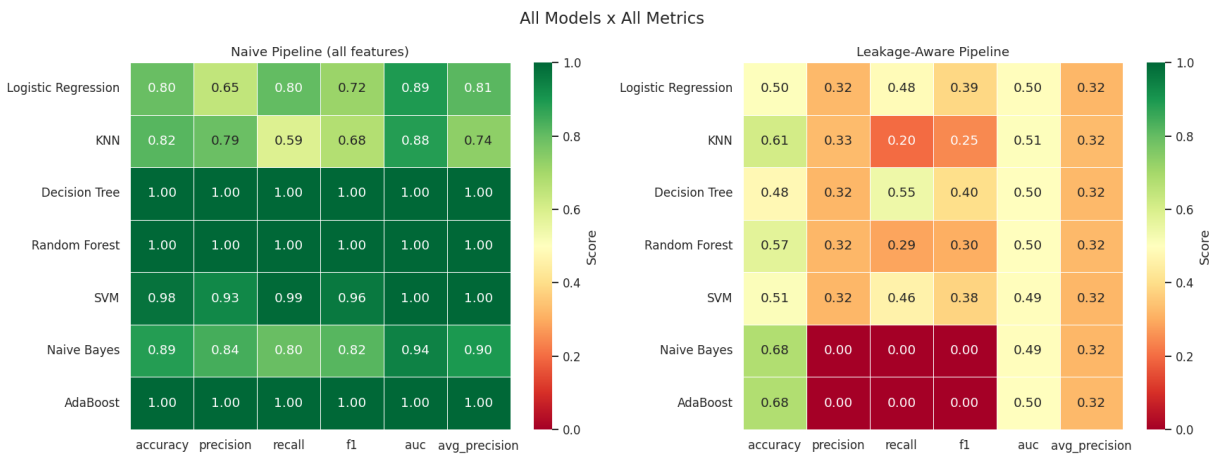


Figure 13. All seven models against all six metrics, naive (left) vs. leakage-aware (right), as a single color-coded heatmap.

11. Cost-Sensitive Threshold Optimization

Every leakage-aware model sits at chance level, so there is nothing genuine to threshold-tune there; that finding is reported as-is above rather than manufactured into a fake optimization. To demonstrate the cost-sensitive thresholding methodology itself on a model with real, graded, learnable signal, this section uses the naive pipeline's Logistic Regression, which (unlike the tree ensembles) cannot fully represent the two-feature rule and so produces smooth, spread-out probabilities rather than a near-perfect step function.

A cost model is defined using each transaction's own dollar amount: a missed fraud (false negative) costs that transaction's real Transaction_Amount; a false alarm (false positive) costs a flat \$5 operational review fee. Sweeping the classification threshold from 0.01 to 0.99 and totaling cost at each point gives:

Threshold	Total Cost
Default (0.50)	\$108,355.62
Cost-optimal (0.18)	\$23,651.86
Savings	\$84,703.76

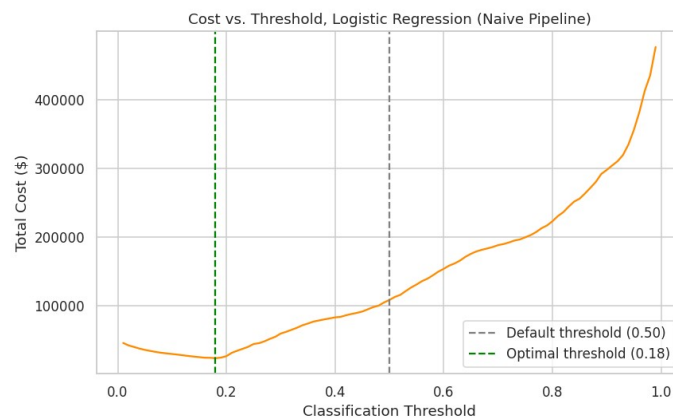


Figure 14. Total cost vs. classification threshold, naive Logistic Regression.

This dollar figure demonstrates the thresholding method only, it is not a trustworthy deployment estimate, since this Logistic Regression still has access to the two leak-derived features, so part of its discriminative power is a shortcut that would not exist in a genuinely leakage-free dataset. To check this honestly rather than merely stating it, the identical sweep was repeated on the leakage-aware Logistic Regression:

Threshold	Total Cost
Default (0.50)	\$266,410.18
"Optimal" (0.01)	\$50,900.00

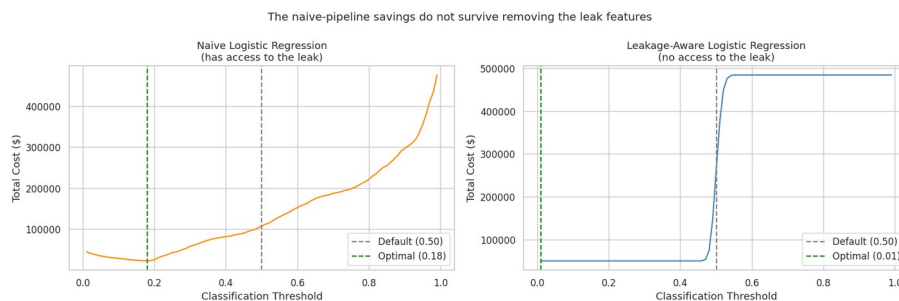


Figure 15. The naive-pipeline cost curve (left) vs. the leakage-aware cost curve (right): the leakage-aware curve does not have a genuine minimum, it monotonically favors flagging almost everyone.

At its own "optimal" threshold, the leakage-aware model flags 100% of all transactions as fraud. With a near-chance model and a false-negative cost (real dollar amounts) far exceeding the flat \$5 false-positive cost, the cost-minimizing policy degenerates to reviewing everyone rather than genuine discrimination, a different failure mode from the naive model's inflated savings, but equally not a usable fraud policy: it is brute-force review, not detection. This comparison is the empirical proof that the naive pipeline's \$84,703.76 figure should not be read as a realistic production estimate.

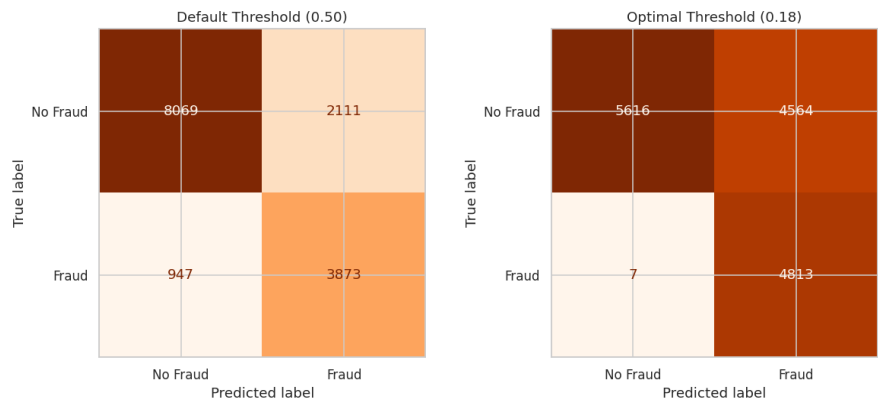


Figure 16. Confusion matrices at the default (0.50) vs. cost-optimal (0.18) threshold, naive Logistic Regression.

12. Explainability with SHAP

Random Forest is used for SHAP analysis (TreeExplainer is fast and exact for tree ensembles), computed on both pipelines for direct comparison, using a random 1,000-row sample of the test set (standard practice for a summary plot, keeping runtime reasonable without changing the conclusion).

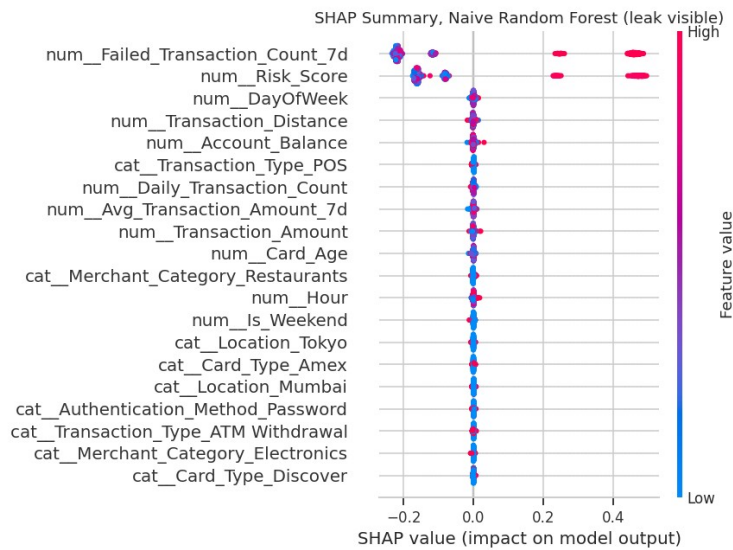


Figure 17. SHAP summary, naive Random Forest: Failed_Transaction_Count_7d and Risk_Score visibly dominate every other feature combined.

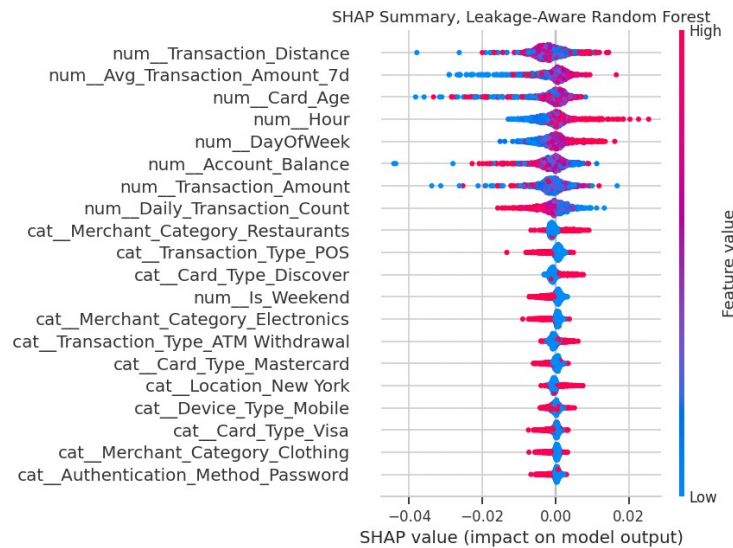


Figure 18. SHAP summary, leakage-aware Random Forest: no feature stands out, consistent with the chance-level performance in Section 9.

These two plots are the clearest single piece of evidence in the whole project. In the naive model, the two leaking features overwhelm every other input combined. Once they are removed, nothing replaces them, confirming visually what Section 9's metrics already showed numerically: there is no meaningful signal left to explain.

13. Conclusion and Key Takeaways

This project set out to compare seven classifiers on a fraud detection task, and ended up demonstrating something more important than any single model's accuracy: the value of auditing a dataset for target leakage before trusting its results.

- Two features, Risk_Score and Failed_Transaction_Count_7d, near-perfectly reconstruct the fraud label on their own (a two-feature rule reaches 100.00% accuracy across all 50,000 rows).
- Tree-based models (Decision Tree, Random Forest, AdaBoost) exploit this instantly, reaching a literal 1.000 on every metric, not because they are the best algorithms for this task, but because they are the best at finding a shortcut.
- Once the two features are removed, every one of the seven models collapses to chance-level ROC-AUC (0.49 to 0.51) and PR-AUC at the no-skill baseline. This synthetic dataset contains no learnable fraud signal outside of the leak.
- Cost-sensitive threshold optimization, applied honestly, extends this finding rather than working around it: the same technique that finds meaningful savings on the naive (leaky) model degenerates to a brute-force "flag everyone" policy on the leakage-aware model, further confirming there is no real signal to optimize against.
- SHAP explainability visually corroborates the numeric findings at every stage: the two leak features dominate the naive model's attributions; nothing replaces them in the leakage-aware model.

The practical lesson generalizes well beyond this dataset: a model's evaluation metrics are only as trustworthy as the process used to audit its inputs for shortcuts. A model reaching 100% accuracy should prompt suspicion before celebration, and the correct response is not to report the number, but to investigate it, exactly as this project did.

14. Limitations and Future Work

- This analysis is specific to this synthetic dataset; the leakage found here is a property of how the data was generated, not a general finding about fraud detection or about any of the seven algorithms used.
- No cross-validation or hyperparameter search was performed; results reflect a single 70/30 stratified split and default (or lightly regularized) hyperparameters.

- The cost model in Section 11 uses an illustrative flat \$5 false-positive cost; a real deployment would calibrate both costs against actual operational and reputational figures.
- For a genuinely leakage-free dataset, the natural next step would be to package the winning model's fitted preprocessor and classifier into a single scikit-learn Pipeline object for serialization and deployment, which was deliberately not done here so as not to obscure the naive-vs-leakage-aware comparison behind an extra layer of abstraction.